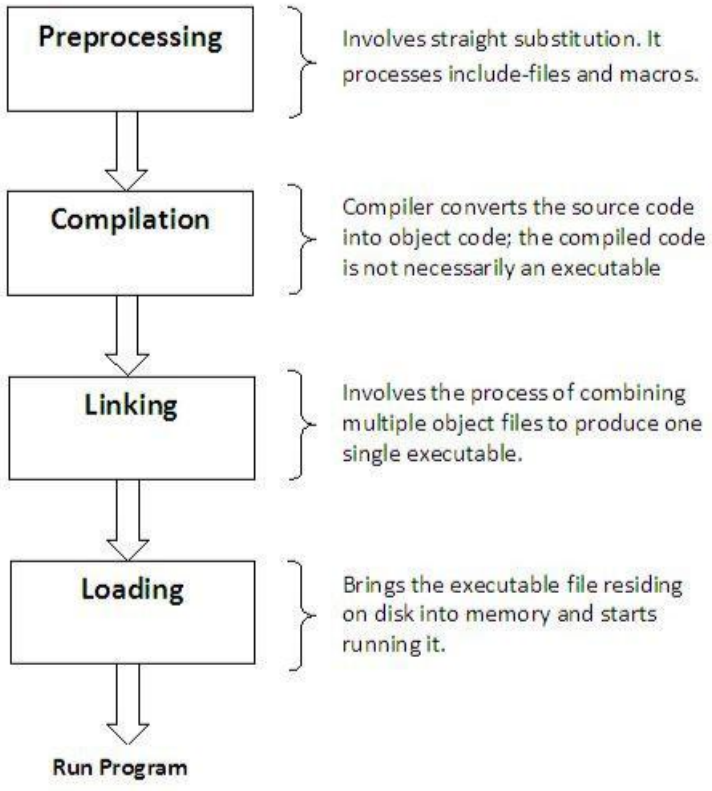
2020-21_CST8234 – C Programming

Fundamentals in C programming

C Compilation process

Important to everyone to understand. This topic will be tested in each midterm and in the final exam.



Compilation process

- **Preprocessor**: Obeys commands that begin with # or directives.
- **Compiler**: Translate a program into a machine instructions or object code
- **Linker**: Combines the object code produced by the compiler with any additional code needed to yield a complete executable.

Preprocessing:
All `#includes` are resolved in this phase: The files are copied into your program.
If the file is included with `< >`, the the original location of the file is `/usr/include`

To see the code generated by the preprocessing:

```
gcc -E -P code.c
```

Errors
You should be able to recognize error coming from the compiler or the linker.

Error from the compiler are about syntax. For example, missing a semicolon or a bracket.

```
gcc -o 02_integers 02_integers.c
02_integers.c: In function 'main':
02_integers.c:10:2: error: expected ';' before 'scanf'
```

Line number where the error is

The error is coming from the `.c` file, the translation to machine-code has not been finalized yet.

Error from the linker are about undefined references. For example, misspell library-function names

```
gcc -o 02_integers 02_integers.c
/tmp/ccKUnOhb.o: In function `main':
02_integers.c:(.text+0x57): undefined reference to `scan'
collect2: ld returned 1 exit status
```

The error is coming from a `.o` file. The **ld** (or linker) reports that did not finish well.

Compilation options

`-Wall`	Causes the compiler to produce warning message when it detects possible errors
`-W`	Issues additional warning message beyond those produced by -Wall
`-pedantic`	Issues all warnings required by the C standard. Causes programs that use nonstandard features to be rejected
`-ansi`	Turns off features of GCC that aren't standard C and enables a few standard features thar are normally disable

GCC: Gnu Compiler Collection

GCC is free and capable of compiling several languages, including C, C++, Java, Objective-C, etc It runs under many operating systems and generates code for many different CPUs. GCC is the primary compiler for UNIX-based OS including Linux, BSD and Mac OS X.

ANSI C

ANSI C refers to the family of successive standards published by the American National Standards Institute (ANSI) for the C programming language. Software developers writing in C are encouraged to conform to the standards, as doing so aids portability between compilers. – Wikipedia

There are three standards today: C89, C99 and the latest C11.

In this class I will be using **C89**.

A Small C program.

Review of `hello.c` as a basic program

First program done in the lab: `0_Numbers.c`

```
/*****************************************************************************/
/* Declare include files
 *****************************************************************************/
#include <stdio.h>
#include <stdlib.h>

/*****************************************************************************/
/* File defines

#define MIN 1
#define MAX 100

/*****************************************************************************
 * Main
 *****************************************************************************/
```

`#define` Preprocessor directive

Q. What is the difference between using a `#define` and declaring `MIN & MAX` as variables?

`main()` should return an `int`

```
int main( void ) {

      int i;

      for ( i = MIN; i <= MAX; i ++ ) {
            if ( !(i % 3) && !(i % 5) ) printf( "%d --> I'm multiple of 3 && 5!!!\n", i);
            else if ( !(i % 3) ) printf( "%d --> I'm multiple of 3!!!\n", i);
            else if ( !(i % 5) ) printf( "%d --> I'm multiple of 5!!!\n", i);
            else printf("%d\n", i );

      }

      return EXIT_SUCCESS;
}
```

`( i%3 )` : returns 0, if `i` is divisible by 3

`!( i%3 )`: Turns it to 1 (true)

`EXIT_SUCCESS` defined as 0 in `stdlib`

Important to have clear the logic and the order of the **ifs** to be tested.

Return vs exit

`return` returns from the **current** function call. If the `expression` of the `return` statement does not match the function return type, the expression will be explicitly converted to the return type. If a non-void function reaches the end of its body – does not execute a return – the behaviour of the program is undefined if it attempts to use the value returned by the function.

`exit` is a system call that terminates the current process. `exit()` is defined in `stdlib`. The parameter of this function is always a integer number, usually 0 to indicate success (or the constant `EXIT_SUCCESS`) and a non-zero value to indicate an error (or the constant `EXIT_FAILURE`). When using `exit` within a function, you should be aware that if you have allocated memory, this memory does not get released. Use always the `atexit()` function to register a function that will clean up your memory.

```
#include <stdio.h>

char func( );
int main( void ) {

      char c;

      printf("Before Calling func()\n");
      c = func( );
      printf("After Calling func(),
        with return value %c\n", c );

      return 0;
}

char func( ) {
      printf("Executing func()\n");
      return 'A';
}
```

returns a char

```
#include <stdio.h>
#include <stdlib.h>

char func( );

int main( void ) {

      char c;

      printf("Before Calling func()\n");
      c = func( );
      printf("After Calling func(),
            with return value %c\n", c );

      return 0;
}

char func( ) {
      printf("Executing func()\n");
      exit( EXIT_SUCCESS );
      printf("I will never print!\n");
      return 'A';
}
```

Needs to include `stdlib.h`

`exit()` finishes the process.
The function does not execute completely.
It never returns to `main()`

C Functions – An introduction

Building blocks from which programs are constructed. Two categories, those written by the programmer and those provided as part of the C implementation, or **library functions.** The `main( )` function is the only mandatory function in a C program.

Function return values

- A function may not return an array, there are no other restrictions on the return type
- A return type of `void` indicates that the function does not return a value
- If the return value is omitted in C89, the function is presumed to return a value of type `int`.

```
#include <stdio.h>
#include <stdlib.h>

double average( double a, double b) {
   return ( a + b ) / 2;
}

int main( void ) {

      double x, y, z;

      printf("Enter three numbers: ");
      scanf("%lf%lf%lf", &x, &y, &z);
      printf("Average of ( %g, %g, %g ) = %g\n", x, y, z, average( average( x, y
), z ) );
      return EXIT_SUCCESS;
}
```

03_average.c

Function Call

A function call consists of a function name followed by a list of arguments enclosed in parentheses.

```
average( x, y );
```

A call of a non-void function produces a value that can be stored in a variable, tested, printed or used in some other way.

Function Declaration or Function prototypes

Functions are usually defined after the `main( )` or sometimes in different files. Move the `average( )` function definition from **`03_average.c`** to after the `main( )` and compile again.

```
root@bahrs:/20F_CST8234/# gcc -o 03_average 03_average.c -Wall -ansi
-pedantic 03_average.c: In function 'main':
03_average.c:48:2: warning: implicit declaration of function 'average'
[-Wimplicit-function-declaration]
03_average.c:48:2: warning: format '%g' expects argument of type 'double', but
argument 5 has type 'int' [-Wformat]
03_average.c: At top level:
03_average.c:58:8: error: conflicting types for 'average'
03_average.c:48:56: note: previous implicit declaration of 'average' was here
```

When the compilers encounters the first call of `average( )` in `main( )`, it has not information about `average( )`: it does not know how many parameters it has, what type of parameters are, or what type it returns.

The compiler,

- Assumes that the function returns an integer!
- Makes an `implicit declaration of a function`.
- Is unable to check that we are passing the right amount of parameters, and the arguments are the right type.

A function prototype provides the compiler with an early view of the function whose full definition will appear later. Function prototypes are placed before the `main( )`

```
double average( double a, double b );
```

Function prototypes does not have to specfy the name of the parameters as long as their type are present:

```
double average( double , double  );
```

It's usually best not the omit the names, as it helps to document what the function does.

Q. So, why not to put all the function definitions BEFORE the `main( )`?

This approach assumes that the function is just going to be called in the `main( )` function. Functions will call each other all the time, then you will need to be careful that in the order in which you define your functions. Or you may want to separate your programs in multiple files.

Prototypes are often in a separate header file which can be included in other C source files that wish to use the functions. The header **`stdio.h`**, for example, contains prototypes for the functions `scanf( )` and `printf( )`. Therefore, our examples have been including this file, so that the compiler will let us call those functions. The actual code for these functions is kept in a library elsewhere on the system, the header file only says how to interface with the functions.

Arguments

Function arguments are **passed by value** in C. When the function is called, each argument is evaluated, and its value assigned to the parameter of the function. Any changes made to the parameter during the execution of the function does not affect the argument. Each parameter behaves like a variable that has been initialized to the value of the matching argument.

A function should do **one and only one thing**. It usually returns a value that then is used by another function.

```
int exponent( int a, int b ) {

	int c = 1;

	while( b-- )
		c *= a;
	return c ;
}
```

The following function intents to interchange two values:

```
void swap( int a, int b ) {

      int temp;

      temp = a;
      a = b;
      b = temp;

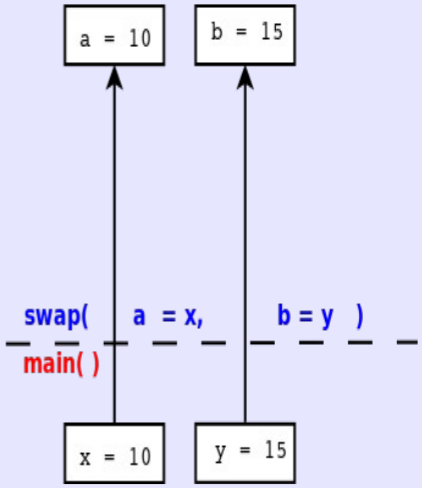
      return;
}

int main( void ) {

      int x = 10, y = 15;

      printf("Before swap x=%d, y=%d \n", x, y );
      swap( x, y );
      printf("After  swap x=%d, y=%d \n", x, y );

      return EXIT_SUCCESS;
}
```



Q. What are the end values for `x` and `y`? Why

Passing arguments by value sometimes makes it difficult to write functions.

Formatted input and output `printf` & `scanf`

```
#include <stdio.h>

int  printf(const char *format, ...);
int fprintf(FILE * stream, const char *format, ...);
```

print formatted. `printf()` formats a string accordingly to an arbitrary number of varied data type parameters. This string is by default printed on the `standard output`. The function returns the number of characters printed, or a negative number in case of an error.

Both functions prototype end with the … (ellipsis symbol) which indicates a variable number of arguments. `printf( )` can have none or multiple parameters passed to it.

```
/* No parameters */
printf( "Hello world\n" );

/* 5 is replaced where the %d is located and then the string printed */
printf("This is an number %d\n", 5 );

/* the value of num1 and num2 are replaced where the %d are */
printf("%d + %d = %d\n", num1, num2, num1+num2 );
```

`fprintf( )` writes to the stream indicated by its first argument.

```
fprintf(fp, "Hello World\n");
fprintf(stderr, "Error:  Invalid input\n");
```

`printf( )` conversion specifications

The function requires a format string containing ordinary characters and/or conversion specifications. Ordinary characters are printed as is. Conversion specifications describe how the remaining arguments are to be converted to character for display. A conversion specification contains a `%` character followed by as many as 5 items:

	flags	width	precision	Length modifier	Conversion specifier
%	#0	12	.5	L	g

Flag:

Flag	***meaning***
-	Left-justify within field
+	Numbers produced by signed conversions always begin with + or -
Space	Nonnegative numbers produced by signed conversions are preceded by a space
#	Octal numbers begin with 0, nonzero hexadecimal numbers with 0x
0 (zero)	Numbers are padded with leading zeros up to the field width

Minimum field width

An item that is too small to occupy this number of characters will be padded.
An item that is too large for the field width will still be displayed in its entirety.

Length modifier

Indicates that the item will be displayed as a type that is longer or shorter than what it is.

Conversion specifier

most used specifiers

Specifier	**Meaning**
d, i	Converts an int to decimal form
o,u,x,	Converts an unsigned int to o: base 8 u: base 10 x: base 16, prints a … f as lowercases X: base 16, prints A...F as uppercases
f	Converts a double to decimal form
e, E	Converts a double to scientific notation
g, G	Converts a double to either f form or e form
c	Displays an int as an unsigned character
s	Displays the characters pointed by the argument. Stops when it reaches a NULL or the number of bytes specified by the precision is reached.
p	Converts a void * to printable form

```
#include <stdio.h>

int scanf(const char *format, ...);
int fscanf(FILE * stream, const char *format, ...);
```

The `scanf()` function reads input from the standard input stream `stdin ( STanDard INput – The keyboard )`

```
int num1;
scanf( "%d", &num1 );
```

The `%d` indicates that the number to read is an integer. Please notice that the variable name has an ***ampersand*** in front. It indicates that the memory location of the variable is being passed, instead a copy of the variable.

On success, the function returns the number of items of the argument list successfully filled. Since assignment into variables stops when given invalid input for a certain format specifier, this can tell you if you've input all your data correctly. The functions return the number of data items that were read and assigned to objects, they return `EOF` if an input failure occurs before any data items can be read.

The following loop test scanf return value, it reads one integer at a time stopping when an input error happens:

`while( scanf( "%d", &i ) ) …`

IMPORTANT
Forgetting to put the `&` in front of the variable in a `scanf` call will be unpredictable!

`scanf` is a very powerful but very unforgiving function. Many programmers avoid the use of it. If the user does not enter the expected input, some programs would not work correctly, unless you do a careful testing of multiple cases, for a short program, doing so its impractical, so in most of the examples done in class, the testing is omitted.

How `scanf` works

When `scanf` is called, it starts processing the information in the string starting at the left. For each conversion specification, `scanf` tries to locate an item of the appropriate type in the input data, skipping blank spaces if necessary. If the item was read successfully, it continues processing the rest of the string. If not, it returns immediately without looking at the rest of the format string.

`scanf` ignores white-characters.

```
#include <stdio.h>
#include <stdlib.h>

int main(  ) {

      int val, num;
      char c;

      /* Read the number */
      val = scanf( "%d%c", &num, &c );

      /* Error check */
      if ( val == 0 )  {
            printf("Invalid format\n");
            exit ( EXIT_FAILURE );
      }
```

```
        else if ( c != '\n') {
              printf("Extra characters\n");
              exit( EXIT_FAILURE );
        }
        else printf("Number read %d\n", num );

        exit( EXIT_SUCCESS );

}
```

Typical Four-Part Algorithm Structure

Special Note:

This was not discussed in class, but it is an important read

By: Ian Allen et. al.

Functionally, most Unix run-once commands (not servers) typically have this form:

1) Input Stage
- verify the correct number of command-line arguments
- optionally prompt for any missing arguments
- complain about missing or extra arguments and exit non-zero

2) Validation Stage
- validate the input: make sure numbers are numbers, files are files, directories are directories, etc.
- test to make sure arguments are not empty or null strings
- test to make sure things exist and are accessible
- test to make sure things are readable/writable

3) Processing Stage
- run your algorithm on your validated input
- solve the problem; generate the results
- servers may stay at this stage "forever" until killed

4) Output Stage
- display the output

Do not mix up code among the four stages. Keep them distinct:

Do not duplicate your Processing Stage or Validation Stage for various combinations of Input arguments. Where possible, collect all the input first; then Validate it *once*, then write your Processing code *once*.

Do not duplicate your Output Stage for various combinations of Input arguments. Generate your results and display the results *once*, at the end of the program. Where possible, don't duplicate similar kinds of output statements all over your code. Write one output statement that uses variables to contain the variable part of the output.

Less code is better code.

Some Programming techniques

Special Note:

This was not discussed in class, but it is an important read

* Correct documentation
  - internal documentation (labelling, function headers, etc.)
  - external documentation (man pages, user guides, testing, etc.)

* Writing clean, simple, robust code: "Less code is better code"!
  - checking the return status of important functions and syscalls
  - choosing the correct loop and branch control structures
  - rewriting the algorithm to minimize duplicated code
  - making it right before you make it clever or fast

* Using correct indentation, style, and mnemonic names
  - variable and file names reflect their use in the algorithm

* Using correct modularization and scope (i.e. local vs. global variables)

* Correct use of arguments and/or prompting for user input
  - some programs will prompt the user for input if no arguments are given
  - some programs will read standard input if no file names are given

* Correct use of exit codes on success/failure of the program

* Presenting accurate and useful help if user makes an error:
  1. error messages must appear on standard error, not on standard output
  2. error text must contain the name of the program issuing the error (use argv[0] or $0, never hard-code the name)
  3. always echo the input that is in error back to the user, and
  4. give a hint as to the valid input that was expected, e.g.:
     "myprog: Age 23 is not between 12 and 18"